

Simulador de Tráfego em Malha Viária



Trabalho 2 - 65DSD

Professor: Fernando dos Santos

Alunos:

Clara dos Santos Becker

Lucas Gabriel Falcade Nunes



Introdução sobre Threads

- *Threads* é uma linha, ou **fluxo de execução** de instruções;
- Objetivo é **aproveitar o processador**, melhor **uso de recursos** e melhor **experiência do usuário**.
- **Cenário do trabalho:**
 - Desenvolver um **simulador de tráfego**, onde veículos se movem por vias de uma malha viária.
 - Cada **veículo é um *Thread***, e devem manter e respeitar o fluxo da pista.

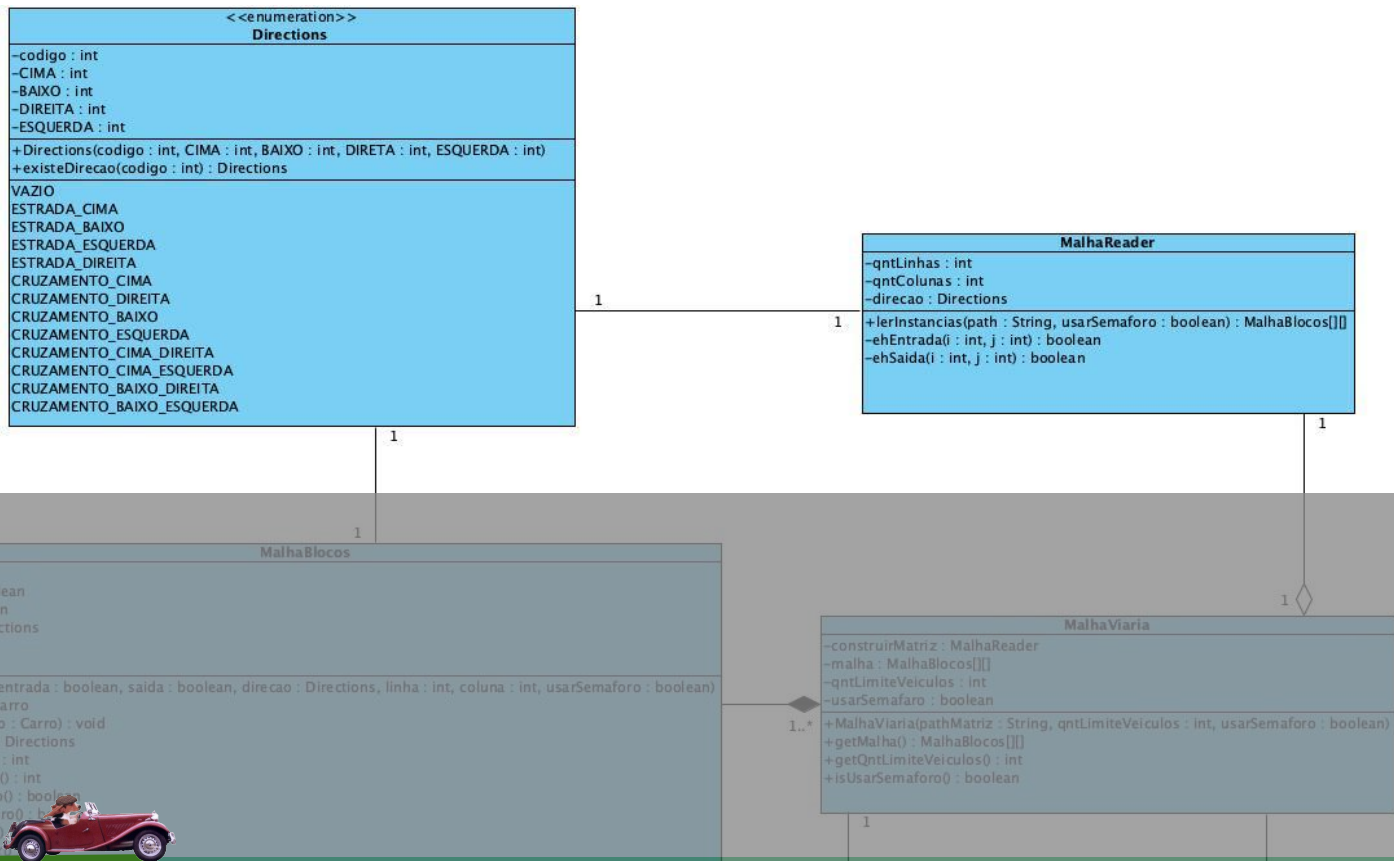


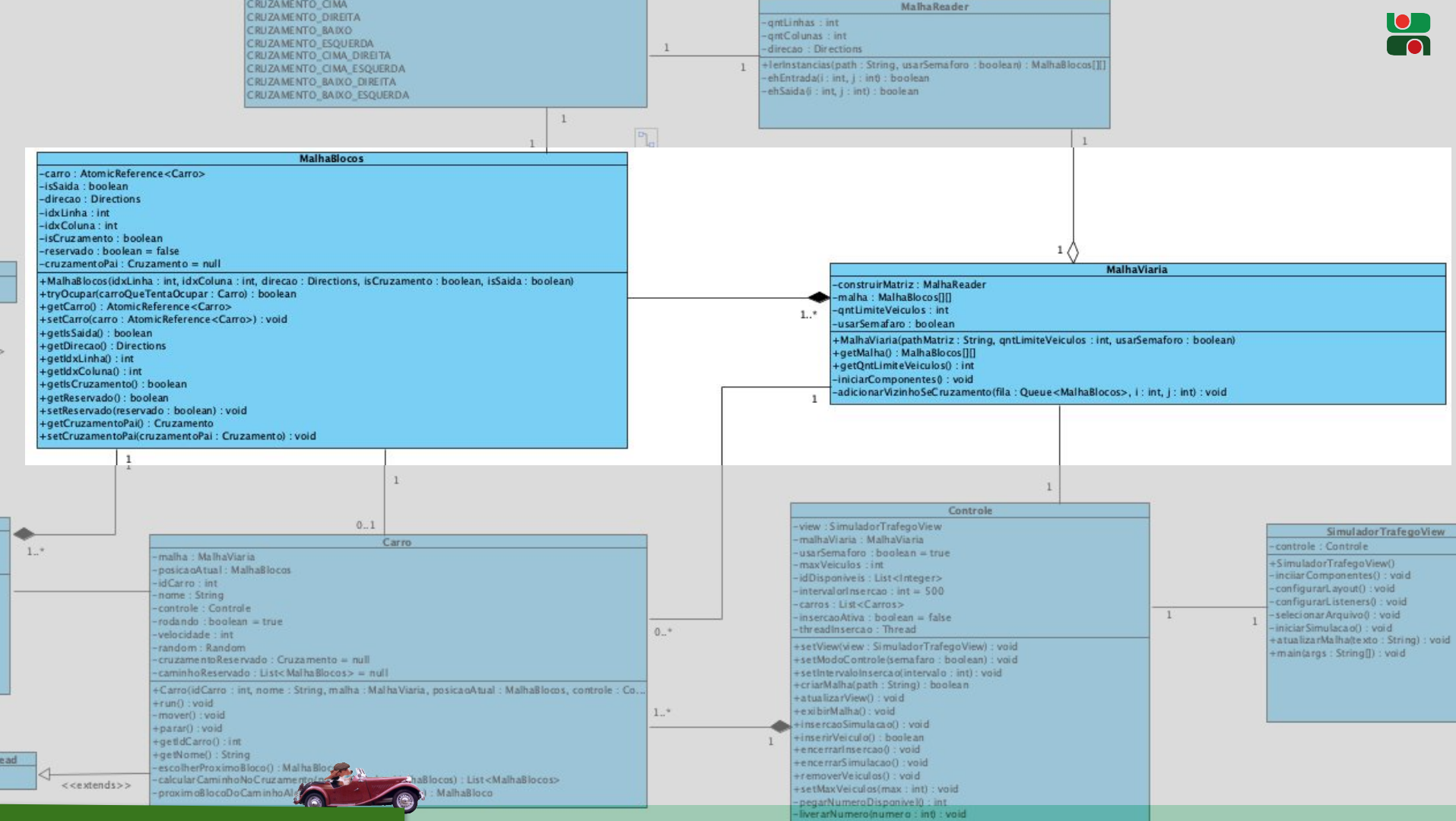


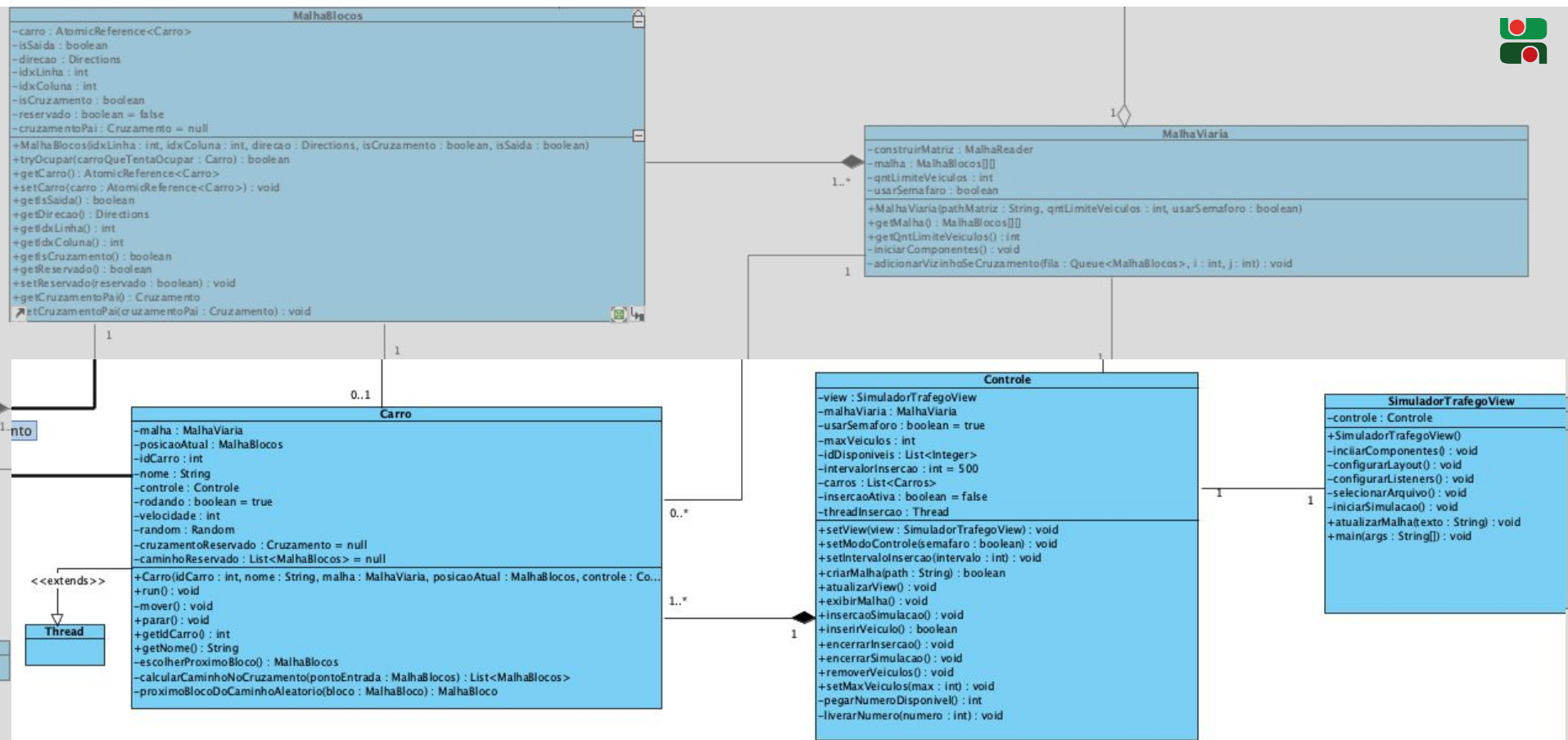
Diagrama de Classes

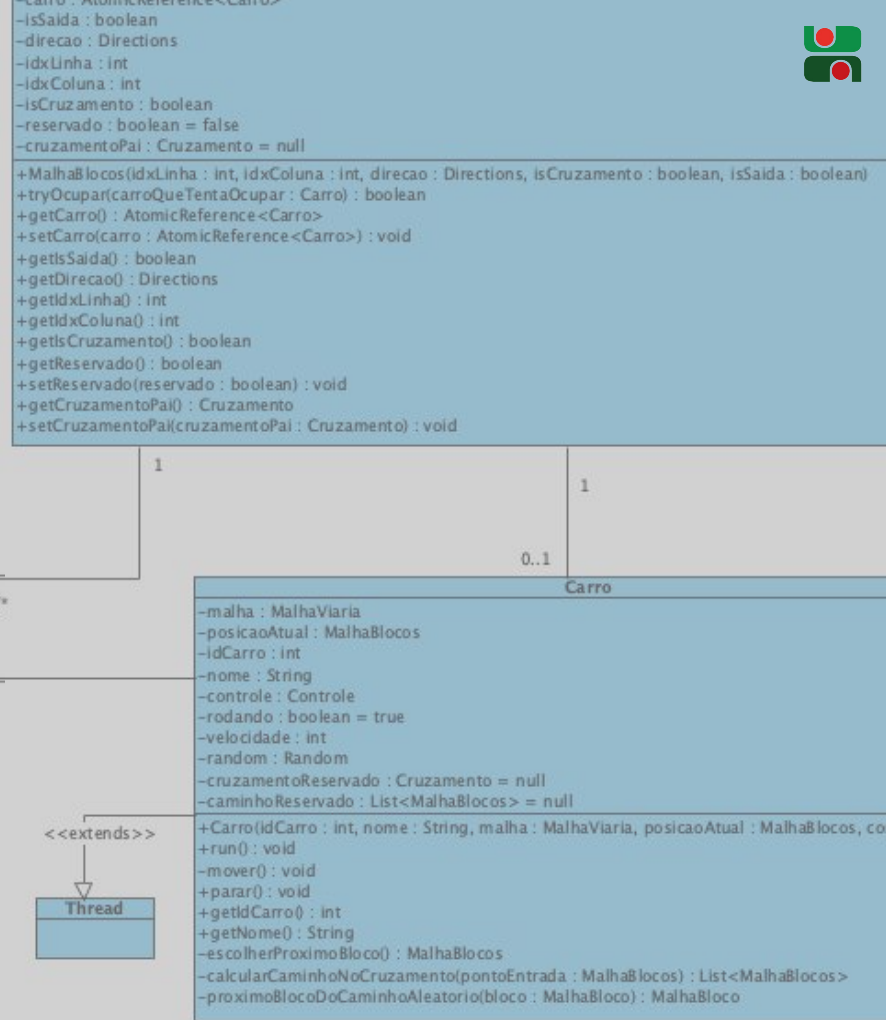
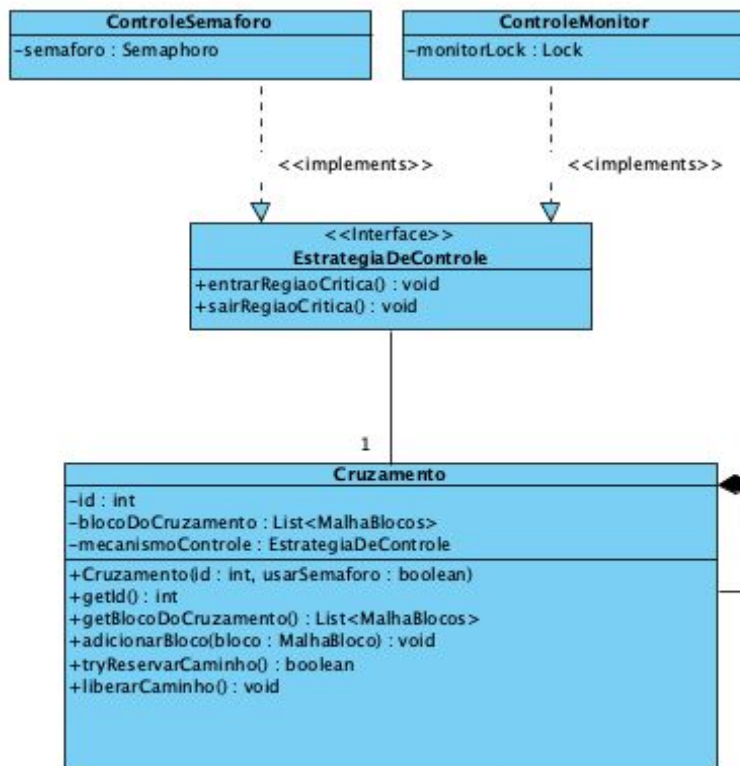












Estruturação

```
> docs
> instances
✓ src
  ✓ main
    ✓ java
      ✓ controller
        © Controle
      ✓ io
        © MalhaReader
      ✓ model
        ✓ estrategia
          © ControleMonitor
          © ControleSemaforo
          ⓘ EstrategiaDeControle
          © Carro
          © Cruzamento
          © MalhaBlocos
          © MalhaViaria
        ✓ util
          ⓘ Directions
      ✓ view
        © SimuladorTrafegoView
```

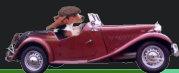




Classe Carro

- Estende *Thread*, permitindo exceção paralela dos veículos;

```
@Override
public void run() {
    try {
        while (rodando) {
            mover();
            controle.exibirMalha();
            Thread.sleep(velocidade);
        }
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
        System.out.println(nome + " foi interrompido e finalizado.");
    }
}
```





```
private void mover() throws InterruptedException {  
    MalhaBlocos proximoBloco;  
  
    //Se o carro já tem um CAMINHO RESERVADO, carro CONTINUA por esse caminho, REMOVENDO O PROXIMO BLOCO DA LISTA  
    if (caminhoReservado != null && !caminhoReservado.isEmpty()) {  
        proximoBloco = caminhoReservado.remove(index: 0);  
    } else {  
        // O CARRO ESCOLHE O PROXIMO LIVRE para MOVER  
        proximoBloco = escolherProximoBloco();  
    }  
  
    //Se não tem bloco disponível  
    if (proximoBloco == null) {  
        // Se cruzamento reservado ele o libera  
        if (cruzamentoReservado != null) {  
            System.out.println(nome + " sem movimento - liberando cruzamento " + cruzamentoReservado.getId());  
            cruzamentoReservado.liberarCaminho(); //liberar caminho  
            cruzamentoReservado = null; //limpa referencia  
        }  
        return;  
    }  
}
```





```
//----- Verifica se BLOCO é Cruzamento -----  
if (proximoBloco.isCruzamento() && cruzamentoReservado == null) {  
    //Obtem a qual bloco o cruzamento pertence  
    Cruzamento cruzamento = proximoBloco.getCruzamentoPai();  
  
    //Espera ativa para gerar logs --> TENTA RESERVAR O CRUZAMENTO  
    while (!cruzamento.tryReservarCaminho()){  
        System.out.println(nome + " AGUARDANDO cruzamento " + cruzamento.getId());  
        Thread.sleep( millis: 50);  
    }  
  
    //Reserva obtida com sucesso  
    cruzamentoReservado = cruzamento;  
  
    //Calcula caminho que o carro deve seguir dentro do cruzamento  
    caminhoReservado = calcularCaminhoNoCruzamento(proximoBloco);  
    System.out.println(nome + " RESERVOU cruzamento " + cruzamento.getId());  
  
    //O próximo bloco agora será o primeiro dentro do cruzamento  
    proximoBloco = caminhoReservado.remove( index: 0);  
}
```





```
//----- MOVIMENTO DO CARRO -----  
//Tenta ocupar o proximo bloco  
if (proximoBloco.tryOcupar( carroQueTentaOcupar: this)) {  
    MalhaBlocos blocoAnterior = posicaoAtual;    //Guarda o bloco anterior  
    posicaoAtual = proximoBloco;    //Atualiza a posicao do carro  
    blocoAnterior.setCarro(null);    //Libera o bloco anterior i.e o carro saiu do bloco (proximo carro pode ocupar)  
  
    //Se o carro saiu do cruzamento e terminou o caminho reservado  
    if (caminhoReservado != null && !posicaoAtual.isCruzamento() && caminhoReservado.isEmpty()) {  
        System.out.println(nome + " LIBEROU caminho do cruzamento " + cruzamentoReservado.getId());  
        cruzamentoReservado.liberarCaminho();    //libera o cruzamento  
  
        //limpa as referencias  
        this.cruzamentoReservado = null;  
        this.caminhoReservado = null;  
    }  
}  
  
//Verifica se é uma saida  
if (posicaoAtual.isSaida()) {  
    parar();    // Para o run do thread  
    posicaoAtual.setCarro(null);  
    controle.removeVeiculo( carro: this);  
}  
}
```





Classe MalhaViaria

```
public class MalhaViaria {  
  
    1 usage  
    private final MalhaReader construirMatriz = new MalhaReader();  
    8 usages  
    private MalhaBlocos[][] malha;  
    2 usages  
    private final int qntLimiteVeiculos;  
    2 usages  
    private final boolean usarSemaforo;  
  
    1 usage  🧑 clarabecker +1  
    public MalhaViaria(String pathMatriz, int qntLimiteVeiculos, boolean usarSemaforo) throws IOException {  
        this.qntLimiteVeiculos = qntLimiteVeiculos;  
        this.usarSemaforo = usarSemaforo;  
        this.malha = construirMatriz.lerInstancias(pathMatriz);  
        inicializarCruzamentos();  
    }  
}
```



Classe MalhaBlocos

```
public class MalhaBlocos {

    2 usages
    private final int idxLinha;
    2 usages
    private final int idxColuna;
    2 usages
    private final Directions direcao;
    2 usages
    private final boolean isCruzamento;
    2 usages
    private final boolean isSaida;

    // --- MUDANÇA PRINCIPAL: Usar AtomicReference para garantir a movimentação segura sem locks ---
    3 usages
    private final AtomicReference<Carro> carro = new AtomicReference<>( initialValue: null);

    2 usages
    private boolean reservado = false;
    2 usages
    private Cruzamento cruzamentoPai = null;

    public MalhaBlocos(int idxLinha, int idxColuna, Directions direcao, boolean isCruzamento, boolean isSaida) {
        this.idxLinha = idxLinha;
        this.idxColuna = idxColuna;
        this.direcao = direcao;
        this.isCruzamento = isCruzamento;
        this.isSaida = isSaida;
    }

    public boolean tryOcupar(Carro carroQueTentaOcupar) {
        // Tenta atômicamente mudar o valor de null para o novo carro.
        // Se o valor atual não for null, a operação falha e retorna false.
        return this.carro.compareAndSet( expectedValue: null, carroQueTentaOcupar);
    }
}
```



Classe Controle

- **Gerencia a simulação** → inicia, encerra simulação e encerra inserção de carros.
- **Gerencia a Malha Viária** → cria malha (*path*), define modo de controle (semáforo ou monitor).
- **Gerência Carros** → insere (gerência IDs dos Carros), remove carros e controla quantidade máxima de carros na malha.
- **Gerência parâmetros da simulação** → tempo de inserção de carros, modo de controle dos cruzamentos e quantidade de carros simultâneos.
- **Atualiza a View**





Regiões Críticas

- mover() → Classe Carro

```
//----- MOVIMENTO DO CARRO -----  
//Tenta ocupar o proximo bloco  
if (proximoBloco.tryOcupar( carroQueTentaOcupar: this)) {  
    MalhaBlocos blocoAnterior = posicaoAtual;    //Guarda o bloco anterior  
    posicaoAtual = proximoBloco;    //Atualiza a posicao do carro  
    blocoAnterior.setCarro(null);    //Libera o bloco anterior i.e o carro saiu do bloco (proximo carro pode ocupar)  
  
    //Se o carro saiu do cruzamento e terminou o caminho reservado  
    if (caminhoReservado != null && !posicaoAtual.isCruzamento() && caminhoReservado.isEmpty()) {  
        System.out.println(nome + " LIBEROU caminho do cruzamento " + cruzamentoReservado.getId());  
        cruzamentoReservado.liberarCaminho();    //libera o cruzamento  
  
        //limpa as referencias  
        this.cruzamentoReservado = null;  
        this.caminhoReservado = null;  
    }  
}
```





Regiões Críticas

- `tryOcupar()` → Classe MalhaBloco

```
// --- MUDANÇA PRINCIPAL: Usar AtomicReference para garantir a movimentação segura sem locks ---  
3 usages
```

```
private final AtomicReference<Carro> carro = new AtomicReference<>( initialValue: null);
```

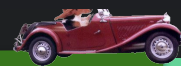
```
public boolean tryOcupar(Carro carroQueTentaOcupar) {  
    // Tenta atomicamente mudar o valor de null para o novo carro.  
    // Se o valor atual não for null, a operação falha e retorna false.  
    return this.carro.compareAndSet( expectedValue: null, carroQueTentaOcupar);  
}
```



Regiões Críticas

- `tryReservarCaminho()`
 - Classe Cruzamento

```
public boolean tryReservarCaminho() {  
    boolean reservado = false;  
    mecanismoControle.entrarRegiaoCritica();  
  
    try {  
        // Verifica se algum bloco está reservado  
        for (MalhaBlocos bloco : blocosDoCruzamento) {  
            if (bloco.isReservado()) {  
                return false; // está ocupado  
            }  
        }  
  
        // Reserva todos os blocos  
        for (MalhaBlocos bloco : blocosDoCruzamento) {  
            bloco.setReservado(true);  
        }  
        reservado = true;  
    } finally {  
        mecanismoControle.sairRegiaoCritica();  
    }  
  
    return reservado;  
}
```





Regiões Críticas

- `inserirVeiculo()` → Controle

```
private final List<Carro> carros = Collections.synchronizedList(new ArrayList<>());

private boolean inserirVeiculo() {
    MalhaBlocos[][] malha = malhaViaria.getMalha();
    // Para evitar ConcurrentModificationException ao iterar, podemos usar uma cópia ou um for indexado
    for (int i = 0; i < malha.length; i++) {
        for (int j = 0; j < malha[i].length; j++) {
            MalhaBlocos bloco = malha[i][j];

            if (bloco != null && ehEntrada(bloco) && bloco.getCarro() == null) {
                int idDisponivel = pegarNumeroDisponivel();
                if (idDisponivel == -1) return false;

                Carro carro = new Carro(idDisponivel, nome: "Carro" + idDisponivel, malhaViaria, bloco, controle: this);
                carros.add(carro);
                carro.start();
                return true;
            }
        }
    }

    return false;
}
```



Semáforos

- Estruturas de controle de acesso
- Gerência quantidade de threads que podem acessar recursos "simultaneamente"

Na classe MalhaBlocos, cada bloco de cruzamento cria um objeto Semáforo.

A thread do carro usa `tryAcquire()` para tentar obter a permissão com um tempo de espera.

Quando o carro sai do cruzamento, ele chama `release()` no semáforo do bloco anterior, liberando a permissão.





Monitores

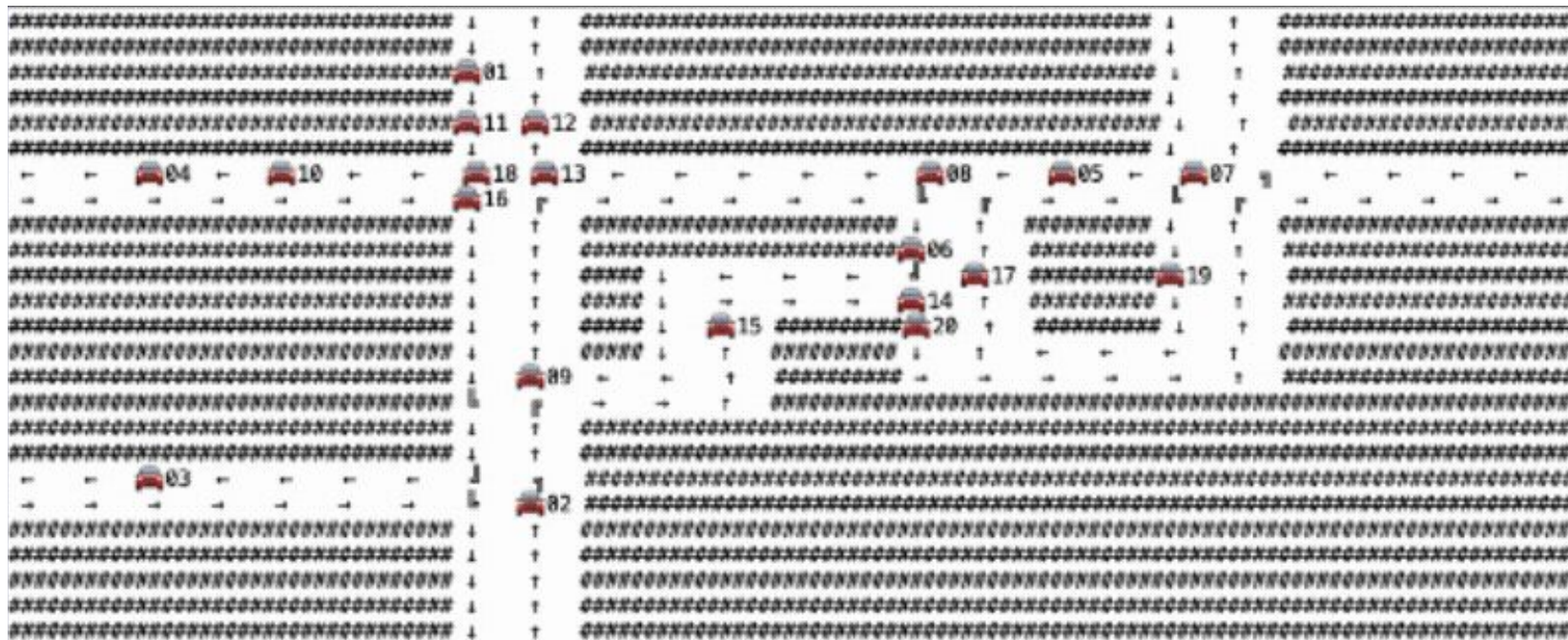
- Mecanismo que garante acesso exclusivo de um recurso compartilhado.
- No Java, cada objeto pode ser um monitor.

Na classe `MalhaBlocos`, o método `entrarMonitor()` é `synchronized`. Dentro do método possui o laço `while` (ocupado) e `monitor.wait()` para fazer a thread esperar de forma eficiente se o recurso estiver ocupado.

O método `sairMonitor()` também é `synchronized`. Ele define `ocupado` como `false` e chama `monitor.notify()`, que “avisa” uma das threads que estava esperando.



Apresentação do Sistema em Execução



*gif demonstrativo





Pontos Importantes

- **Uso de sincronização** na malha **previne que 2 carros** ocupem o mesmo bloco (MalhaBloco) simultaneamente.
- Simulador suporta os **2 tipos de controle** de cruzamento:
 - **Semáforo**: controle dos **blocos críticos de forma limitada**;
 - **Monitor**: garante **exclusão mútua nos blocos**, evitando colisões ou "atropelamento".

Limitações:

- Controle limitado para maiores quantidade de carros → **Redução do paralelismo** em **altas densidades**.
- **Interface Gráfica** deformada devido ao formato texto (JTextArea).





Conclusão

- O trabalho mostrou uma implementação dos conceitos de *threads*, semáforos e monitores.
- Apresenta **soluções** e mostra a **complexidade** do uso de sistemas com *threads*.
- Apresenta a facilidade de gerar "gargalos" em um sistema.
- A **importância** para do uso na **Engenharia de Software**.



OBRIGADO!

Momento aberto para dúvidas

